

Novelty Library WebApp — Product Requirements Document

Source inspected: NL_WA_1_75 (uploaded archive NL_WA_1_35.zip)

Document status: Current-state PRD + target requirements

Product: Novelty Library

1. Product summary

Novelty Library is a reader-focused digital book-review and discovery platform. It combines a curated review archive, reader-submitted reviews, community reactions, reader profiles, shareable profile cards, analytics, social features, editorial/admin workflows, and monetisation infrastructure.

The current implementation is a React/Vite single-page web application backed by Supabase. The product is intentionally editorial rather than a generic review marketplace: the platform owns the review lifecycle, gives reviewers attribution, exposes reader/community feedback on published review posts, and provides an admin-controlled publication pipeline.

2. Product goals

1. Make high-quality book discovery easier through a searchable/browsable review shelf.
2. Give reviewers a credited, durable home for their opinions.
3. Keep the distinction between the reviewer's R/W Rating and the community's cumulative NL Rating explicit.
4. Let registered readers build a configurable Novelty Library identity/profile.
5. Provide profile-card generation suitable for social sharing and downloading.
6. Give administrators a controlled editorial pipeline from submission through acceptance and publication.
7. Provide transparent platform analytics without exposing private user data.
8. Allow selected public-facing copy and profile-question structure to be managed without code changes.
9. Support monetisation and operational modules without coupling them tightly to the reading experience.

3. Target users

3.1 Public visitor

- Browse the homepage and review shelf.
- Read published reviews.
- View public profiles.
- Read About, Privacy, Terms, Cookies and review guidelines.
- View platform analytics.
- Follow external purchase/poster/blog links where available.

3.2 Registered reader

- Everything a visitor can do.
- Maintain a Novelty Library profile.

- Answer configurable profile questions.
- Upload permitted profile images.
- Follow/unfollow readers and manage follower/following privacy.
- Publish 24-hour stories.
- Like, rate and review published community review posts.
- Save review drafts.
- Submit reviews.
- Reserve a book where the workflow supports it.
- Receive notifications.
- Manage account/security settings.

3.3 Reviewer/contributor

- Submit a review with book metadata, ratings and review text.
- Receive a post-submission editing window.
- Track accepted/published reviews through profile/account surfaces.
- Receive attribution on published content.

3.4 Administrator

- Manage review submissions and editorial status.
- Manage accepted/published/community review records.
- Manage posters and publication metadata.
- Manage reserved books.
- Manage users and admin access.
- Manage site support contact and finance dashboard links.
- Edit public page copy.
- Manage review guidelines.
- Configure profile questions, profile sections, core-field labels/visibility/order, and profile-card presentation.
- Access the Supabase dashboard.

4. Core product entities

- **User/Auth account:** Supabase Auth identity.
- **Profile:** reader identity, avatar/header, social links, reading journey, privacy, profile answers.
- **Profile question:** admin-defined question with type, options, limits and profile-card behavior.
- **Review:** submitted editorial object with book/reviewer metadata, R/W rating and lifecycle status.
- **Master list record:** publication-oriented canonical review record.
- **Community review:** published review surface used by readers for likes/ratings/comments.
- **Feedback:** a reader's rating/review/tags against a published review post.
- **Book reservation:** reservation/claim workflow for a book.
- **Story:** temporary 24-hour reader content.
- **Notification:** user-facing activity alert.
- **Editable page:** database-driven public/admin copy.

- **About section:** structured About-page content.
- **Finance module:** admin-managed external finance/payment dashboard link.

5. Functional requirements

5.1 Home

- Present Novelty Library identity and primary navigation.
- Show live library metrics supplied by `home_analytics()`.
- Provide entry points to Reviews, Submit, About and Analytics.
- Render a 300x250 sponsored Adsterra placement where configured.
- Support the story rail and reader-oriented promotional/engagement surfaces.
- Preserve responsive and reduced-motion behavior.

5.2 Review discovery

- Show published review cards.
- Support genre filtering.
- Open individual review pages through stable hash routes.
- Show book cover/poster, metadata, review text, reviewer attribution and ratings.
- Preserve external poster/purchase links when present.

5.3 Review detail

- Display reviewer R/W Rating as the reviewer's opinion.
- Display Goodreads/Amazon ratings when present.
- Display NL Rating only from community-submitted ratings for that review post.
- Never merge the reviewer's R/W score into the NL community average.
- Support likes, reader ratings, reader reviews and comments.
- Support authenticated editing/deletion of the current user's feedback.
- Sanitize user-provided display text and external URLs.

5.4 Review submission

Required/core submission flow includes:

- Book title.
- Author.
- Genre.
- Language.
- Traits.
- Review text.
- Goodreads rating.
- R/W rating.
- Undertaking/acceptance checkbox.

Additional functionality includes:

- Open Library lookup.
- Cover selection/upload.
- Duplicate/book availability warning.
- Honeypot anti-spam field.
- Quality guidance.
- Draft saving/loading.
- Book reservation workflow.
- Post-submit edit window of five minutes while the submitted state remains active.
- Submission progress indicator.

5.5 Review lifecycle

The product must maintain a clear state model:

pending -> approved/accepted -> published

A declined review remains declined and must not become public accidentally.

Publication may create/update the corresponding master-list and community-review records. Poster and publication metadata are managed separately from initial submission data.

5.6 Profiles

Registered users can configure:

- Name.
- Avatar.
- Header image.
- Instagram and up to three social links in the current implementation.
- Reading since.
- Books read this month.
- Total books read.
- Favourite book.
- Favourite author.
- Favourite genre.
- Configurable profile-question answers.
- Follower/following privacy.
- Profile visibility settings.

Name and account email are treated as locked core fields for admin field configuration.

5.7 Profile questions

Admins can configure:

- Short answer.
- Paragraph.
- Number.
- Year.

- URL.
- Single select.
- Multiple select.
- Image upload.

Configurable properties include:

- Required status.
- Public default visibility.
- Profile-card inclusion.
- Profile-card mode: tag, question + answer, or answer-only.
- Maximum selections for multi-select.
- Alphabetical option sorting.
- Image count, size and resolution limits.
- "Other" support for select questions.
- Section and order.

5.8 Profile cards

The profile card is a visual exportable reader identity artifact.

Supported export formats:

- 9:16 — 1080x1920.
- 16:9 — 1600x900.
- 3:4 — 1200x1600.
- 4:5 — 1080x1350.
- 1:1 — 1080x1080.

Requirements:

- The visible preview should correspond to the exported canvas.
- Export uses `html-to-image`.
- Content should progressively scale/trim when it overflows.
- Published reviews appear as a reader shelf.
- Ratings use the platform's star representation.
- The 4:5 format receives a dedicated editorial composition.
- 16:9 uses a split composition.

5.9 Community/social

- 24-hour stories.
- Story likes/reposts.
- Reader follows.
- Follower/following privacy.
- Notifications.
- Community review likes.
- Community review ratings.

- Community reader reviews/comments.

5.10 Analytics

The public analytics layer can expose aggregate information such as:

- Accepted/published totals.
- Average form rating.
- Average book rating.
- Reviewer count.
- Author count.
- Language counts.
- Top contributor.
- Recent accepted items.
- Top-rated accepted items.
- Community likes/ratings/reviews.
- Total users.

Private contact/email data must not be exposed through public analytics.

5.11 Administration

Admin access is determined by the backend `is_admin()` function.

The admin hub currently exposes:

- Book Reviews.
- Admins & Contacts.
- Users & Accounts.
- Finances and Payments.
- WebApp Pages Moderation.
- Supabase Dashboard shortcut.

The admin review area is consolidated around submitted/accepted/published/reserved/poster/community workflows.

5.12 Content management

Admins can edit:

- Review guidelines.
- About sections.
- Privacy policy.
- Terms.
- Cookie policy.
- Cookie banner copy.
- Profile question structure.
- Profile section layout.
- Core profile field labels/placeholders/default visibility/order.

6. Non-functional requirements

- Responsive from mobile through desktop.
- Accessible labels and keyboard-operable controls where practical.
- Respect prefers-reduced-motion.
- Sanitize user text before rendering.
- Validate external URLs before navigation.
- Never expose service-role credentials in client code.
- Use one Supabase client instance throughout the browser.
- Gracefully degrade non-critical analytics/engagement integrations.
- Avoid making the public reading experience dependent on admin-only systems.
- Preserve visual consistency in light and dark themes.

7. Success metrics

Primary:

- Published reviews per month.
- Review submission completion rate.
- Review-to-publication conversion rate.
- Returning readers.
- Community engagement per published review.
- Profile completion rate.
- Profile-card generation/share activity.

Secondary:

- Story participation.
- Follow graph growth.
- Average community rating participation.
- Ad impressions/revenue.
- Draft recovery rate.

8. Product principles

1. Reader-first, not algorithm-first.
2. Attribution is part of the product, not an afterthought.
3. Reviewer opinion and community opinion remain separate measurements.
4. Admin configurability must not make the reader experience feel administrative.
5. Visual polish should communicate literary/editorial quality.
6. Public aggregate analytics should be useful without becoming a privacy leak.
7. Graceful degradation is preferable to blocking the core reading journey.